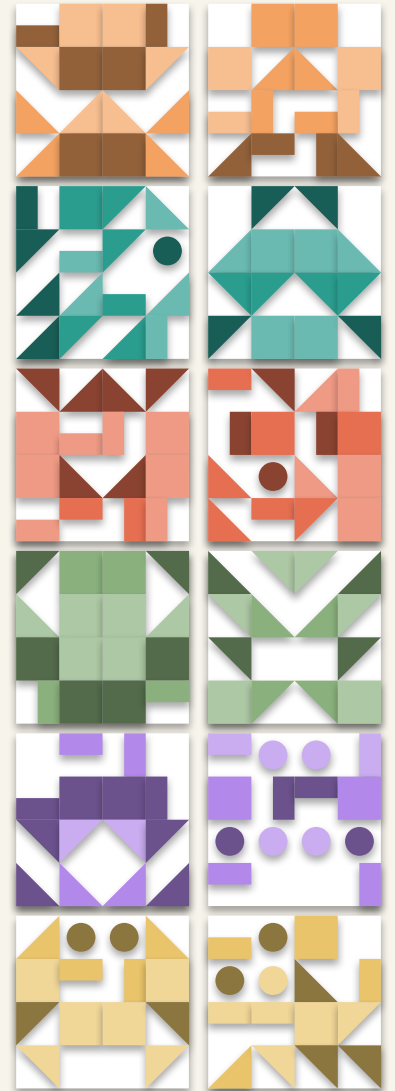


The Treasurer Is a Smart Contract

Rebuilding the stokvel with Solidity and Investec
Programmable Banking

Investec holds the rand. The chain holds the rulebook. The relay carries messages and can't decide.

Nevin Tom data engineer · MSc student · builds on Programmable Banking



A group chat, December

fictional · every name in this talk is made up

Guys, where's December's money? 😐

Priya · 09:41

Guys?

Priya · 09:58

...

Thabo is typing

Every stokvel has
this thread.

One spreadsheet, one
banking app, one person.

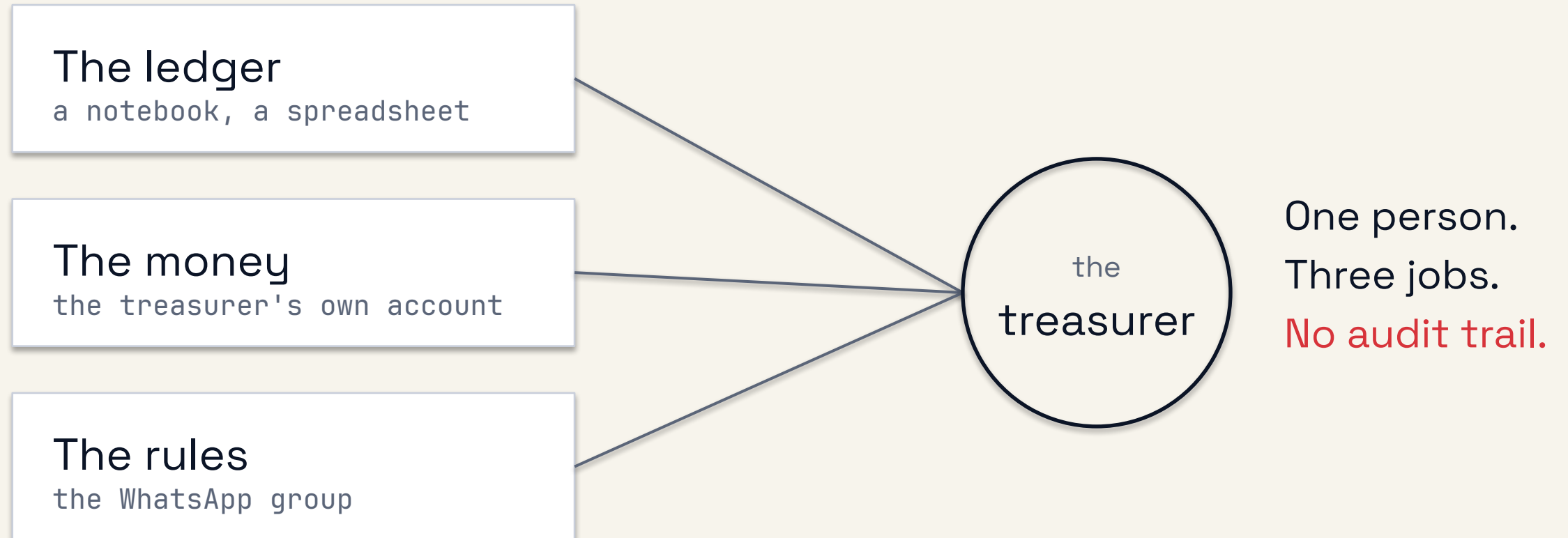
11 million 810,000 R50bn

members

stokvels

saved a year

Fifty billion rand a year, moving on trust.



The pattern

money stays in the bank · rules live on chain · the relayer is a messenger only

Money

Investec

holds every rand
pays members through the API
the sandbox on stage, production in life

Rules

Stokvel.sol

who paid, whose turn, what is owed
refuses what the rules refuse
public, cheap to read, never holds funds

Messages

the relayer

carries facts both ways
signs nothing that decides anything
kill it and restart it, nothing is lost

What the messenger cannot do

CANNOT

pick a recipient

there is no setter; the order is the order

CANNOT

repeat a payment

seen[ref] makes every Investec id single use

CANNOT

invent reserves

the attester is a different key

CANNOT

outvote the members

votes are card signatures, counted per member

“We didn’t replace the trust. We wrote it down where nobody can erase it.”

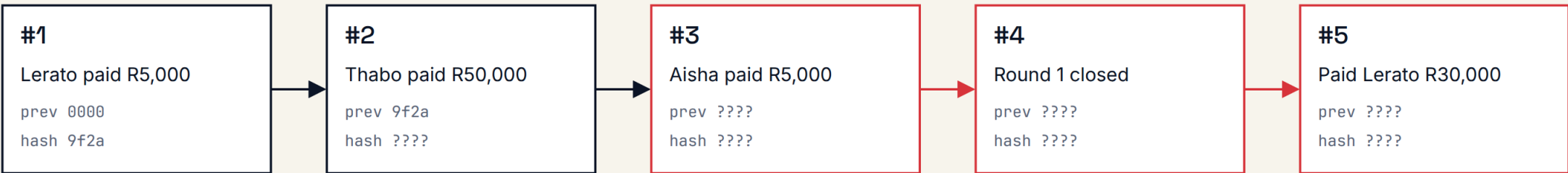
Blockchain in 60 seconds

each block carries the hash of the one before it

Honest



Change one number in block #2



BROKEN

Every block's hash covers the block before it. Rewrite one line and every later hash stops matching.

Thousands of computers hold the same chain, so the rewrite would have to fool all of them at once. That is what "nobody can erase it" means.

DEMO 1

The committee

Six volunteers, six cards, one constitution. Nobody types a name in: the chain knows each member from a signature.



DEMO 2

Swipe and round-up

Aisha buys R87 of bread. The card rounds up to R90 and R3 lands on chain with her number on it.



```
function closeRound() external {
    uint256 owed = pot + unsettled;
    if (reservesAt == 0 || reservesAt + RESERVES_MAX_AGE < block.timestamp) {
        revert ReservesStale(reservesAt, notBefore);
    }
    if (reservesCents < owed) revert ReservesShort(reservesCents, owed);
    if (block.timestamp < roundEndsAt) revert RoundStillOpen(roundEndsAt);

    uint8 recipient = uint8(round % members.length);
    uint256 cents = pot;
    payouts[round] = Payout(recipient, cents == 0, cents, members[recipient].beneficiaryHash);
    pot = 0;
    unsettled += cents;
    emit PayoutDue(round, recipient, cents, members[recipient].beneficiaryHash);

    round += 1;
    roundEndsAt = uint64(block.timestamp) + roundLength;
}
```

No function takes a recipient. `round % 6`, always.

DEMO 3

Month end

The contract says who. Investec pays through the API. The chain records the bank's own payment reference.



DEMO 4

The treasurer steals

R1,000 leaves the account with the label "Admin fees". Watch the payout freeze.



```
function recordContribution(uint8 memberId, uint128 cents, bytes32 investecRef)
    external onlyRelayer
{
    if (seen[investecRef]) revert AlreadyRecorded(investecRef);
    if (memberId ≥ members.length) revert UnknownMember(memberId);
    if (cents > MAX_CONTRIBUTION_CENTS) revert AmountTooLarge(cents, MAX_CONTRIBUTION_CENTS);

    seen[investecRef] = true;
    members[memberId].paidCents += cents;
    pot += cents;
    totalIn += cents;
    emit ContributionRecorded(round, memberId, cents, investecRef);
}
```

V1 is this function without line 4. Same bank transaction, sent twice: V1 counts it twice, V2 says no.

**REFUSED**

Four invariants, always

- 1 `sum(paidCents) = totalIn`
- 2 `totalIn - settledOut = pot + unsettled`
- 3 no Investec ref is ever used twice
- 4 no close while reserves < owed

10,000 random months: contributions, duplicate webhooks, theft, refunds, time travel, closes, payouts.

```
$ ./scripts/fuzz.sh
```

```
[FAIL: an Investec ref was recorded twice: 2 ≠ 1]  
  contribute(uint8,uint128) args=[156, 2592000]  
  duplicateWebhook(uint256) args=[3]  
[FAIL: totalIn ≠ what the bank received: 1184000 ≠ 592000]  
  
Suite result: FAILED. 3 passed; 2 failed (V1)  
Suite result: ok.      5 passed; 0 failed (V2)
```

Write down what must always be true. Let a machine try to make it false.

DEMO 7

Vote

Four cards sign one English sentence: "raise the contribution from R5,000 to R6,000". Then a timelock. Then it's the rule.



DEMO 8

Stolen card

The thief holds Sipho's key. The committee moves Sipho to a spare card, and the stolen one stops being anyone.



DEMO 9

Kill the server

The relayer dies on stage. The book is still on the shelf: every screen rebuilds from chain events, and nothing is sent twice.



What this doesn't do

Trusted relayer

it reports what the bank did

Group account KYC

six people on one key is a demo

No tokenised rand

yet, on purpose

QR cards

should be passkeys

Next

Passkeys

a phone that signs

A real club account

with the bank's own controls

Rage-quit and late fees

the rules a real stokvel has

Tokenised rand

when a bank stands behind one

We didn't replace the trust.
We wrote it down where
nobody can erase it.



[github.com/Nevvyboi/
StokvelProgrammableBankingWithSolidity](https://github.com/Nevvyboi/StokvelProgrammableBankingWithSolidity)

Thank you, committee.



Lerato



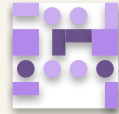
Thabo



Aisha



Johan



Priya



Sipho

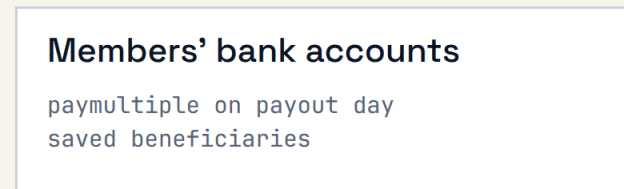
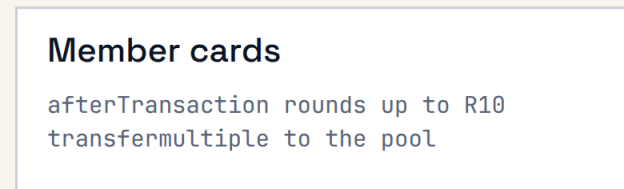
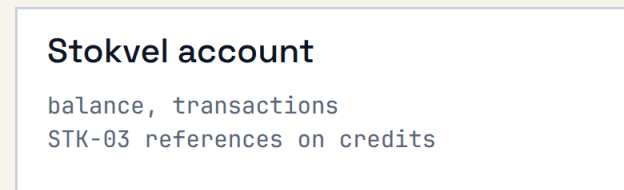
Dodgy treasurer	"pay me this month"	no recipient setter; vote plus timelock	NO SETTER
Dodgy treasurer	takes rand out of the account	ReservesShort freezes payouts	FROZEN
Compromised relayer	records a payment twice	seen[ref]: AlreadyRecorded	REFUSED
Compromised relayer	invents a huge contribution	AmountTooLarge cap; short at close	CAPPED
Compromised relayer	pays the wrong person	beneficiaryHash check; vote it out	CHECKED
Vote thief	replays an old signed vote	EIP-712 domain, per-signer nonce	NONCE
Stolen member card	votes as the victim	RotateKey to a spare card	ROTATED

Architecture

Investec holds the rand • the chain holds the rulebook • the relay carries messages

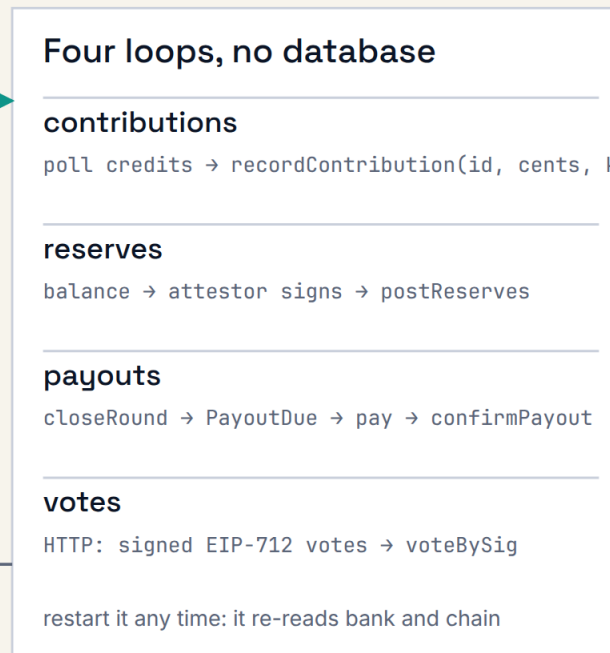
Investec

the money



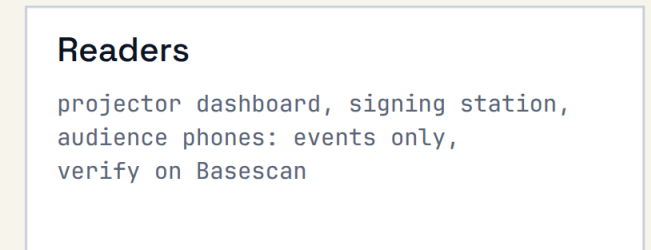
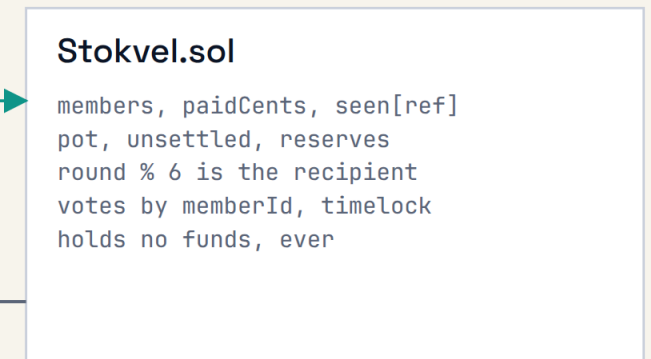
Relayer

the messenger



Chain

the rulebook



Two keys. The relay records what the bank did. The attestor signs what the bank holds. Neither can choose who gets paid.

Sandbox and testnet only. Mock mode mirrors the Investec sandbox on the laptop for the parts the stateless sandbox can't show.